

Robotic Arm Grasping + Multimodal Visual Understanding + SLAM Navigation

Robotic Arm Grasping + Multimodal Visual Understanding + SLAM Navigation

1. Course Content
2. Preparation
 - 2.1 Content Description
 - 2.2 Starting the Agent
3. Running Examples
 - 3.1 Starting the Program
 - 3.2 Test Cases
4. Source Code Analysis

1. Course Content

Basic: After running the example program, combine Nav2 navigation, robotic arm grasping, and AI large model visual understanding to perform complex task functions.

Advanced: Master the key source code introduced in this tutorial.

1. **Note: This chapter requires completing the map mapping file configuration first.**

2. Preparation

2.1 Content Description

This course uses the Jetson Orin NX as an example. For Raspberry Pi and Jetson-Nano boards, you need to open a terminal on the host machine, then enter the command to enter the Docker container. After entering the Docker container, enter the commands mentioned in this course in the terminal. For instructions on entering the Docker container from the host machine, please refer to the "Entering the Robot Car Docker (For Jetson-Nano and Raspberry Pi 5 users)" section in the product tutorial [0. Instructions and Installation Steps]. For Orin and NX boards, simply open a terminal and enter the commands mentioned in this course.

2.2 Starting the Agent

Note: If the agent is already running, there is no need to start it again.

Enter the following command in the vehicle terminal:

```
sh start_agent.sh
```

The terminal will print the following information, indicating a successful connection:

```
Jetson@yahboom: ~  
Jetson@yahboom: ~$ ./open_agent.sh  
[1749538760.063253] Info | TermiosAgentLinux.cpp | init | running... | fd: 3  
[1749538760.064012] Info | Root.cpp | set_verbose_level | logger setup | verbose_level: 4  
[1749538760.941420] Info | Root.cpp | create_client | create | client_key: 0x0DA64EFC, session_id: 0x81  
[1749538760.941537] Info | SessionManager.hpp | establish_session | session established | client_key: 0x0DA64EFC, address: 0  
[1749538760.979971] Info | ProxyClient.cpp | create_participant | participant created | client_key: 0x0DA64EFC, participant_id: 0x000(1)  
[1749538760.983035] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x000(2), participant_id: 0x000(1)  
[1749538760.988244] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x000(3), participant_id: 0x000(1)  
[1749538760.993117] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x000(5), publisher_id: 0x000(3)  
[1749538760.997675] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x001(2), participant_id: 0x000(1)  
[1749538761.001121] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x001(3), participant_id: 0x000(1)  
[1749538761.007277] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x001(5), publisher_id: 0x001(3)  
[1749538761.010634] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x002(2), participant_id: 0x000(1)  
[1749538761.014296] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x002(3), participant_id: 0x000(1)  
[1749538761.020171] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x002(5), publisher_id: 0x002(3)  
[1749538761.023939] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x003(2), participant_id: 0x000(1)  
[1749538761.029173] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x003(3), participant_id: 0x000(1)  
[1749538761.034377] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x003(5), publisher_id: 0x003(3)  
[1749538761.038946] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x004(2), participant_id: 0x000(1)  
[1749538761.042215] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x004(3), participant_id: 0x000(1)  
[1749538761.048422] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x004(5), publisher_id: 0x004(3)  
[1749538761.051600] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x005(2), participant_id: 0x000(1)  
[1749538761.057494] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x005(3), participant_id: 0x000(1)  
[1749538761.062183] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x005(5), publisher_id: 0x005(3)  
[1749538761.066842] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x006(2), participant_id: 0x000(1)  
[1749538761.071217] Info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x0DA64EFC, subscriber_id: 0x000(4), participant_id: 0x000(1)
```

3. Running Examples

3.1 Starting the Program

- On the vehicle terminal, open the terminal and enter the command to start the AI agent function:

```
ros2 launch multi_brains 11m_agent_control.launch.py
```

- Alternatively, you can use the shortcut command:

```
multi_brains
```

After initialization, the following content will be displayed:

```
Jetson@yahboom: ~  
[component_container-1] [INFO] [1749260601.294158664] [camera.camera]: Disable frame sync  
[component_container-1] [INFO] [1749260601.294363626] [camera.camera]: Device DaBai DCW2 connected  
[component_container-1] [INFO] [1749260601.294395179] [camera.camera]: Serial number: AUIMB4200AW  
[component_container-1] [INFO] [1749260601.294418891] [camera.camera]: Firmware version: RD1014  
[component_container-1] [INFO] [1749260601.294439467] [camera.camera]: Hardware version:  
[component_container-1] [INFO] [1749260601.294451499] [camera.camera]: device unique id: 1-2.3.1-9  
[component_container-1] [INFO] [1749260601.294477292] [camera.camera]: Current node pid: 74601  
[component_container-1] [INFO] [1749260601.294489612] [camera.camera]: usb connect type: USB2.0  
[component_container-1] [INFO] [1749260601.294501260] [camera.camera]: Start device cost 1246 ms  
[component_container-1] [INFO] [1749260601.294513900] [camera.camera]: Initialize device cost 912 ms  
[component_container-1] [INFO] [1749260601.614950265] [camera.camera]: Publishing static transform from  
ir to depth  
[component_container-1] [INFO] [1749260601.615063130] [camera.camera]: Translation 0, 0, 0  
[component_container-1] [INFO] [1749260601.615080154] [camera.camera]: Rotation 0, 0, 0, 1  
[component_container-1] [INFO] [1749260601.615097467] [camera.camera]: Publishing static transform from  
color to depth  
[component_container-1] [INFO] [1749260601.615115131] [camera.camera]: Translation 12.317, 0.046452, 1.6  
3189  
[component_container-1] [INFO] [1749260601.615130747] [camera.camera]: Rotation -0.00113323, 0.00116674,  
0.000693509, 0.999998  
[component_container-1] [INFO] [1749260601.615145211] [camera.camera]: Publishing static transform from  
depth to depth  
[component_container-1] [INFO] [1749260601.615158651] [camera.camera]: Translation 0, 0, 0  
[component_container-1] [INFO] [1749260601.615262429] [camera.camera]: Rotation 0, 0, 0, 1  
[asr-5] [INFO] [1749260610.932988522] [asr]: The online asr model :gummy-realtime-v1 is loaded  
[asr-5] [INFO] [1749260610.942192950] [asr]: asr_node Initialization completed  
[model_service-3] [INFO] [1749260611.387760909] [model_service]: LargeModelService node Initialization c  
ompleted...
```

- Start the navigation command on the vehicle:

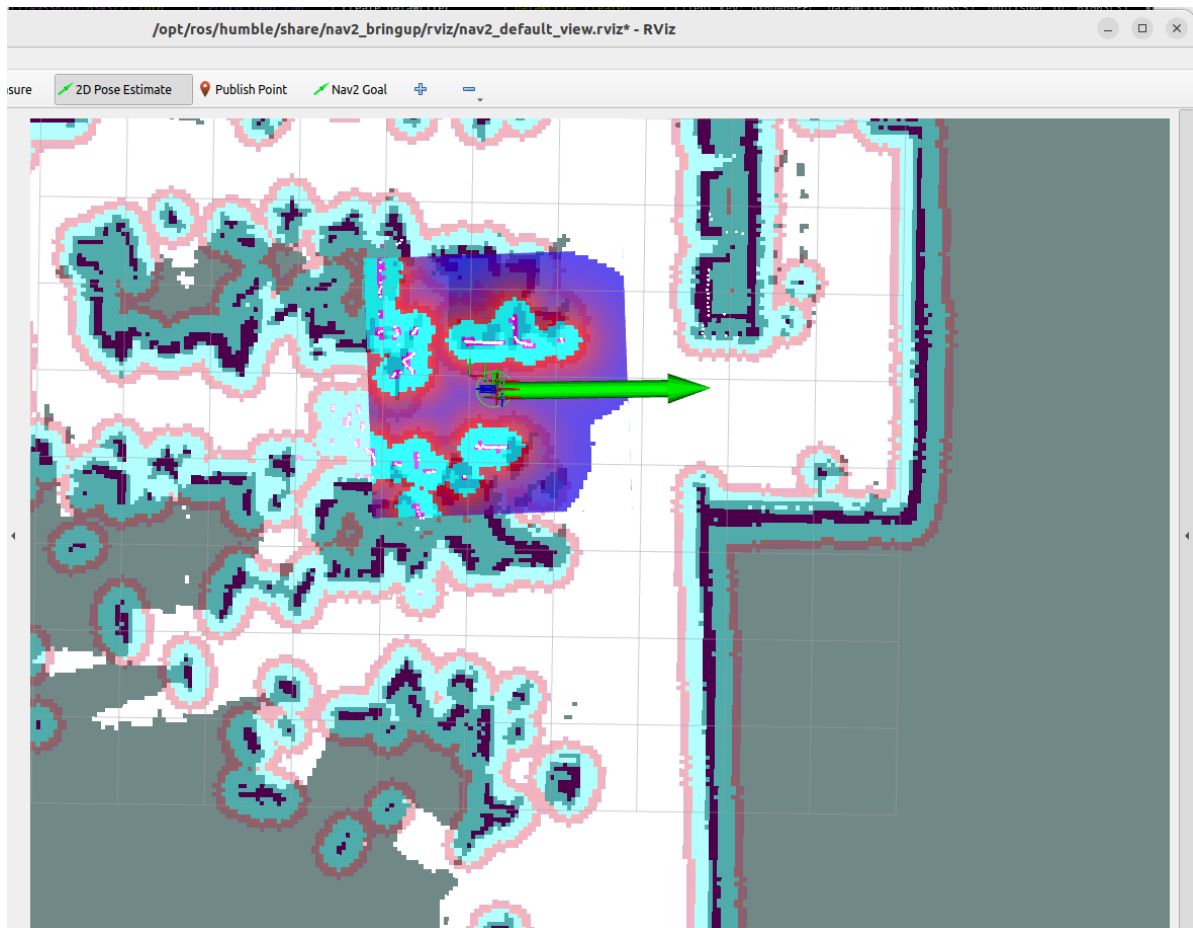
```
ros2 launch M3Pro_navigation base_bringup.launch.py
```

```
ros2 launch M3Pro_navigation navigation2.launch.py
```

- Start rviz on the robot:

```
ros2 launch M3Pro_navigation nav_rviz.launch.py
```

- Then, follow the process for starting the navigation function to initialize the positioning. This will open the rviz2 visualization interface. Click on **2D Pose Estimate** in the toolbar above to enter the selection state. Roughly mark the robot's position and orientation on the map. After initializing the positioning, the preparation is complete.



3.2 Test Cases

Wooden blocks used: 30x30x30 mm blocks.

The test cases here are for reference only; users can create their own dialogue commands.

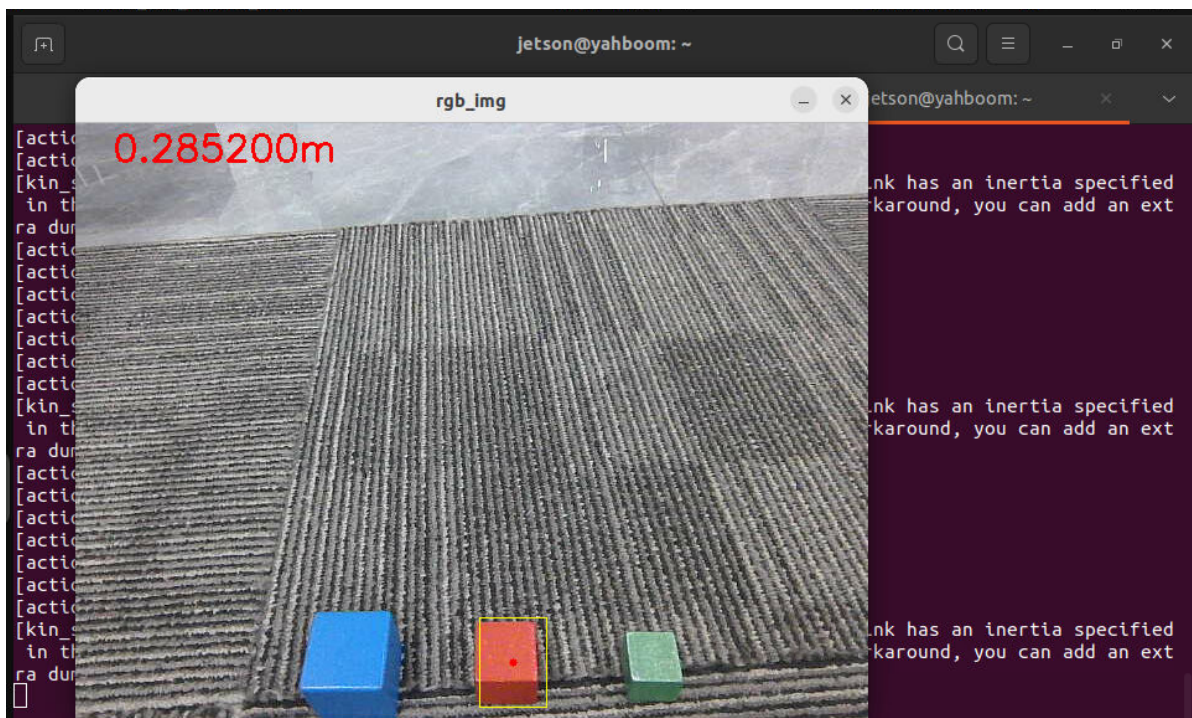
- I am currently in the master bedroom. Please bring the red cube in front of you to the master bedroom.


```
jetson@yahboom: ~  
[asr-5] [INFO] [1749549228.396757797] [asr]: 0  
[asr-5] [INFO] [1749549228.486669892] [asr]: 0  
[asr-5] [INFO] [1749549228.578750722] [asr]: 0  
[asr-5] [INFO] [1749549228.670072394] [asr]: 0  
[asr-5] [INFO] [1749549228.762238058] [asr]: 0  
[asr-5] [INFO] [1749549228.852788924] [asr]: 0  
[asr-5] [INFO] [1749549228.944713333] [asr]: 0  
[asr-5] [INFO] [1749549229.035328393] [asr]: 0  
[asr-5] [INFO] [1749549229.125801048] [asr]: 0  
[asr-5] [INFO] [1749549229.217032830] [asr]: 0  
[asr-5] [INFO] [1749549229.307577808] [asr]: 0  
[asr-5] [INFO] [1749549231.865243587] [asr]: 我现在在主卧室，请你帮我把你面前的红色方块拿到主卧。  
[asr-5] [INFO] [1749549231.866657804] [asr]: okay😊, let me think for a moment...  
[model_service-3] [INFO] [1749549236.016431076] [model_service]: The upcoming task to be carried out: 1.  
调用视觉函数获取红色方块在画面中的坐标  
[model_service-3] 2. 使用机械臂夹取红色方块  
[model_service-3] 3. 导航前往主卧室  
[model_service-3] 4. 放下红色方块  
[model_service-3] [INFO] [1749549238.247663068] [model_service]: "action": ['seewhat()'], "response": 好的，我先看看面前的红色方块在哪里。  
[model_service-3] [INFO] [1749549249.887211674] [model_service]: "action": ['grasp_obj(347,280,396,341)'], "response": 我已经看到了红色方块的位置，现在准备用机械臂夹取它。  
[kin_srv-2] [WARN] [1749549257.735376647] [kdl_parser]: The root link base_link has an inertia specified in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an extra dummy link to your URDF.  
[action_service-4] start  
[action_service-4] [INFO] [1749549261.725444770] [image_converter]: Received: [347, 280, 396, 341]
```

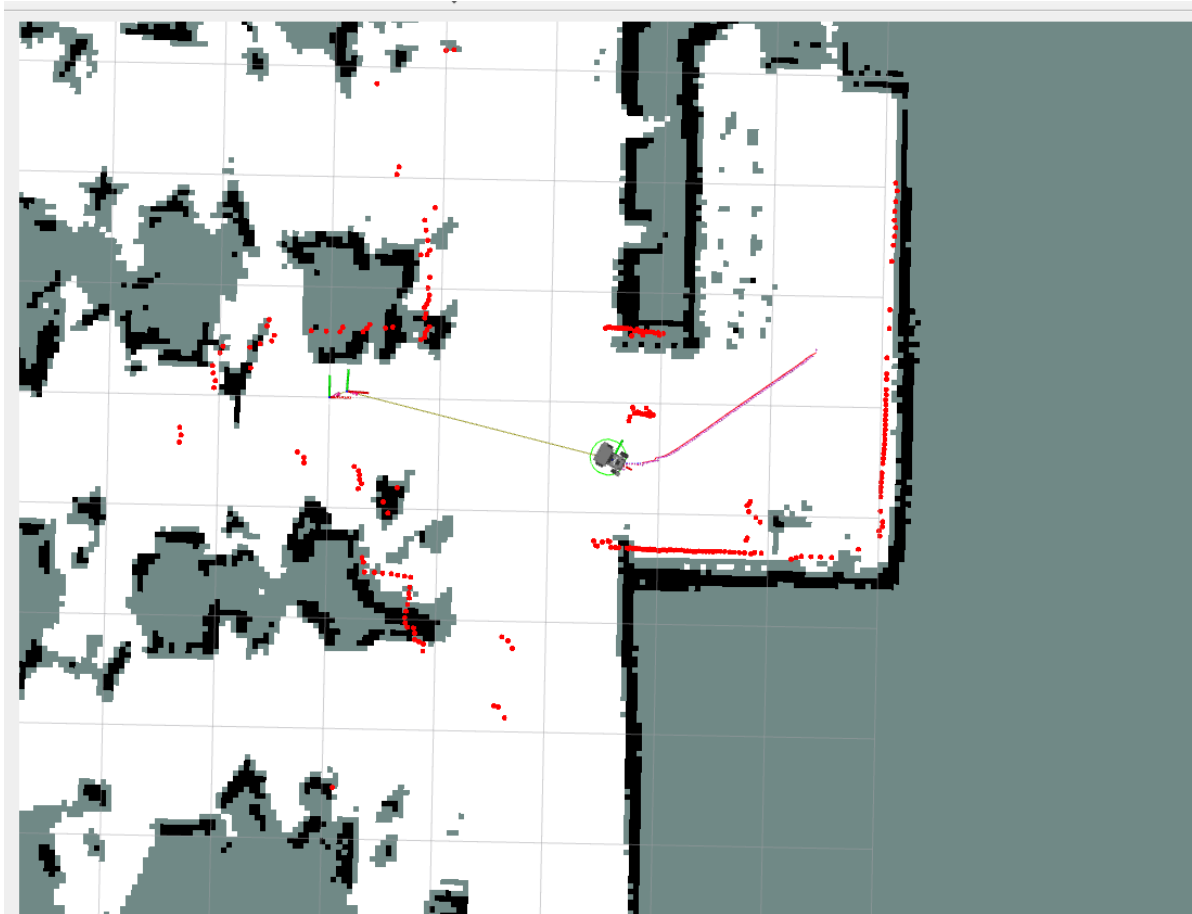
The task steps planned by the decision-making layer model are as follows:

```
[model_service-3] [INFO] [1749549236.016431076] [model_service]: The upcoming task to be carried out: 1.  
调用视觉函数获取红色方块在画面中的坐标  
[model_service-3] 2. 使用机械臂夹取红色方块  
[model_service-3] 3. 导航前往主卧室  
[model_service-3] 4. 放下红色方块  
[model_service-3] [INFO] [1749549238.247663068] [model_service]: "action": ['seewhat()'], "response": 好的，我先看看面前的红色方块在哪里。  
[model_service-3] [INFO] [1749549249.887211674] [model_service]: "action": ['grasp_obj(347,280,396,341)'], "response": 我已经看到了红色方块的位置，现在准备用机械臂夹取它。  
[kin_srv-2] [WARN] [1749549257.735376647] [kdl_parser]: The root link base_link has an inertia specified in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an extra dummy link to your URDF.  
[action_service-4] start  
[action_service-4] [INFO] [1749549261.725444770] [image_converter]: Received: [347, 280, 396, 341]
```

According to the task steps planned by the decision-making layer AI, the robot will first observe the environment in front of it, and then pick up the red block.



Then the robot navigates to the target point along the path planned by the global planner.



After arriving at the navigation target point, the robot will use its robotic arm to put down the red block it is holding and prompt the user that the task is complete. At this point, the robot enters a waiting state (retaining previous conversation memory), and the user can choose to continue the conversation or command the robot to end the current task and start a new task cycle.

```

Jetson@yahboom: ~
jetson@yahboom: ~ x jetson@yahboom: ~ x jetson@yahboom: ~ x
in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an extra dummy link to your URDF.
[kin_srv-2] [WARN] [1749554694.230892821] [kdl_parser]: The root link base_link has an inertia specified in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an extra dummy link to your URDF.
[kin_srv-2] [WARN] [1749554694.236474280] [kdl_parser]: The root link base_link has an inertia specified in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an extra dummy link to your URDF.
[kin_srv-2] [WARN] [1749554694.240666463] [kdl_parser]: The root link base_link has an inertia specified in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an extra dummy link to your URDF.
[kin_srv-2] [WARN] [1749554694.244799797] [kdl_parser]: The root link base_link has an inertia specified in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an extra dummy link to your URDF.
[kin_srv-2] [WARN] [1749554694.250654158] [kdl_parser]: The root link base_link has an inertia specified in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an extra dummy link to your URDF.
[model_service-3] [INFO] [1749554702.868951526] [model_service]: "action": ['navigation(A)'], "response": "红色方块已经夹取成功, 现在我将前往主卧室。"
[action_service-4] [INFO] [1749554708.400056960] [action_service]: navigation Goal accepted
[action_service-4] [INFO] [1749554771.641868476] [action_service]: Navigation succeeded!
[model_service-3] [INFO] [1749554775.480216042] [model_service]: "action": ['putdown()'], "response": "我已经到达主卧室, 现在将红色方块放下。"
[model_service-3] [INFO] [1749554791.990187985] [model_service]: "action": ['finishtask()'], "response": "红色方块已经成功放在主卧室了, 任务完成!"

```

4. Source Code Analysis

- Robot action source code path:

```
~/M3Pro_ws/src/multi_brains/multi_brains/action_service.py
```

- This section's example uses the `seewhat`, `navigation`, `load_target_points`, `putdown`, and `grasp_obj` methods from the `CustomActionServer` class. `seewhat`, `navigation`, `load_target_points`, and `grasp_obj` have already been explained in previous sections. This section will explain the newly introduced `putdown` function.

`putdown` function:

This function controls the robotic arm to release the object it has grasped. After grasping an object, the robotic arm is in a gripping state. Calling this function allows the robotic arm to release the grasped object. The principle is to control the robotic arm's posture by publishing the robotic arm joint topics. A return value of `True` indicates that the action was executed successfully, and the result is fed back to the AI large language model for the next operation.

```
def putdown(self):
    self.pubSix_Arm(self.putdown_joints) # Deployment of robotic arm
    time.sleep(4)
    self.pubSingle_Arm(6, 30, 1000) # The robotic arm opened its gripper and
    released the object.
    time.sleep(3)
    self.pubSix_Arm(self.init_joints) # The robotic arm retracted.
    return True
```